Classes & Objects

→ Object Oriented Programming.

→ Class → a blueprint of creating objects

```
→ class Car:
→     pass
→ audi = Car()
→ bmw = Car()
→ print(audi)
```

⇒ < ~~object~~ __main__.Car object at ... >

→ ~~audi.windows = 4~~
~~print(audi.windows)~~
~~⇒ 4~~

Not the proper way

PTO

Constructor

class declaration

~~class Dog:~~

~~def __init__(name, age):~~

keyword for good practise (can use any name)

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age
dog1 = Dog('Blu', 5)
print(dog1.name)
print(dog1.age)
```

class definition

necessary arguments

object creation

passed arguments

=> Blu
5

Instance Methods

```
class Dog:
    def __init__(self, name, age):
        self.name = name
        self.age = age

    def bark(self):
        print('Woof Awoooooo')

dog2 = Dog('Blu', 5)
dog2.bark()
```

constructor

function declaration

calling class function

=> Woof Awoooooo

default value

eg

```
class BankAcc :
    def __init__(self, owner, balance=0):
        self.owner = owner
        self.balance = balance

    def deposit(self, amount):
        self.balance += amount

    def withdraw(self, amount):
        self.balance -= amount.

    def get_balance(self):
        return self.balance
```

To deduct amount after withdraw

To fetch the current balance

To update new balance after deposit

usage →

```
account = BankAccount('Viv', 500)
print(account.balance)
```

⇒ 500

→ account.deposit(100) → ~~deposits +100~~ +100 balance

→ account.withdraw(300) → -300 balance

→ account.get_balance()

⇒ 300

Inheritance

→ Allows a class to inherit attributes and methods from another class.

eg

```
class Car:
    def __init__(self, window, door):
        self.window = window
        self.door = door.

    def drive(self):
        print('Self driving car')
```

Contr

parent class

Now, for a more advanced blueprint for Tesla,

Parent Class

→

```
class Tesla(Car):
    def __init__(self, window, door, auto, ai):
        super().__init__(window, door)

        self.auto = auto
        self.ai = ai

    def autodrive(self):
        print('Yes')
```

Child Class

These 2 inherit using super().__init__

no : with super()

The drive() is also inherited, & can be accessed by the object of class Tesla.

(*) Multiple inheritance

```
class A:                          ← class name
    def __init__(self, name):
        self.name = name.

    def g_n(self):
        return self.name
```

Base class 1

```
class B:
    def __init__(self, age):
        self.age = age

    def g_n(self):
        return 'yoyo'
```

Base class 2

```
class C(A, B):
    def __init__(self, name, age):
        A.__init__(self, name)
        B.__init__(self, age)

    def g_n(self):
        return 'haha'
```

child class.

parent class name used to call constructors

Note: No colon used. since its a function call

```
x = C('viv', 25)
print(x.g_n())
```

x is an object of C, so the function in C class will take priority

O/P ⇒ haha